Build Your Own Tool

Classwork: Custom Tools + Agent Memory

Name: _____ **Date:** _____

Last week you ran a real agent. Today you UPGRADE it: have your agent build a brand-new tool, then give it a memory that survives a restart. Pick a Driver (shares screen and types) and switch the role each mission so everyone gets a turn!

Before You Start — Setup Check (about 5 min)

Follow the Setup Guide first. Tick these off as a group before Mission A.

1. opencode is open and connected (OpenRouter + DeepSeek V4 Flash).
2. Python is installed (the agent needs it to run your tool).
3. You have a fresh, empty folder open as your project.

> **RECAP**
> Reminder from last week: a tool is just a function the agent can call, and the agent reads the docstring to decide WHEN to use it.

> **NO PYTHON? READ THIS**
> Can't install Python on your computer? No problem! Be your group's ARCHITECT: design the tools and prompts, and have a groupmate run them while sharing their screen. You can ALSO do every Memory mission on your own machine — those need no Python at all.

Mission A — Build a Tool (about 12 min)

You don't have to write Python yourself — describe the tool clearly and let your agent build it!

1. Give your agent this mission (you can swap in your own tool idea):

Copy this prompt exactly:

```
Build me a small MCP server in Python with one tool called roll_dice. It takes a
number of sides and returns a random roll. Give it a clear docstring. Then tell me
exactly how to add it to opencode.json.
```

2. Watch the agent write the code and the opencode.json line. Approve each step.

What did the agent name the tool, and what does it do?

What does the docstring say? (Look in the code the agent wrote.)

Mission B — Use Your Tool (about 10 min)

Plug it in and take it for a spin.

1. Add the agent's opencode.json line if it isn't there yet, then RESTART opencode.
2. Now ask the agent to use your new tool:

Copy this prompt exactly:

```
Roll a 20-sided dice for me three times using your dice tool.
```

Did the agent use YOUR tool? What numbers did it roll?

If it didn't work at first, what fixed it? (Hint: did you restart?)

CONCEPT
You just built and used a tool that didn't exist 10 minutes ago. That's exactly what real developers do!

Mission C — Improve Your Tool (about 10 min)

Real tools get better over time. Make the agent UPGRADE your tool by describing a change. Remember to restart after each change!

1. Pick an upgrade and ask the agent for it (or invent your own):

Copy this prompt exactly:

```
Add a second tool to my dice server called flip_coin that returns heads or tails.
Give it a clear docstring, then remind me to restart.
```

What new tool or feature did you add?

2. Restart, then test the upgraded tool. Try breaking it on purpose!

Ask the agent to roll a 1-sided dice, or flip the coin 10 times. What happened?

CONCEPT
Describing a change and having the agent fix or extend the code is the build-test-refine loop again — now for tools YOU own.

Mission D — Give It Memory (about 12 min)

Now make your agent remember you, even after it restarts.

1. First, test its memory WITHOUT help. Tell it something, then check:

Copy this prompt exactly:

```
My group's name is the Space Cadets and our favorite game is Minecraft. What is our
group's name?
```

Does it know your group's name right now? (It should, in the same chat.)

2. Now have it SAVE that to memory so it survives a restart:

Copy this prompt exactly:

```
Please save our group name and favorite game to your AGENTS.md memory file so you
remember next time.
```

3. RESTART opencode, then ask:

Copy this prompt exactly:

```
What is my group's name and our favorite game?
```

After restarting, did the agent still remember? What did it say?

CONCEPT
Memory is just a file! The agent wrote your facts into AGENTS.md and read them back when it started up. That's long-term memory.

Mission E — Interview Your Agent (about 8 min)

Learn how your upgrades work by asking the agent. Short questions, low token cost. Write its answers in your own words.

Copy this prompt exactly:

```
In 2-3 sentences, explain how the tool I built is different from your built-in
tools.
```

How is your custom tool different from a built-in tool?

Copy this prompt exactly:

```
How do you decide WHEN to use my tool instead of just answering yourself?
```

What did the agent say about how it picks your tool? (Hint: docstring!)

Copy this prompt exactly:

```
Where exactly do you store the things you remember about us?
```

Where does the agent keep its memory?

Mission F — Quick Quests (about 12 min)

Short, fun missions. Do as many as you can — check the box when your group finishes one!

- [ ] Build a tool that turns a word into 'secret code' (e.g. reversed). Test it.
- [ ] Build an emoji tool: give it a mood, get an emoji back.
- [ ] Build a 'random compliment' tool and have the agent cheer up your group.
- [ ] Build a leap-year checker tool: give it a year, find out if it's a leap year.
- [ ] Save THREE new facts to memory (mascot, motto, favorite snack), restart, and check all three.
- [ ] Ask the agent to list every tool it has right now, including yours.
- [ ] Have the agent write a fancier docstring for one of your tools, then test it still works.
- [ ] STRETCH: ask the agent to find and install a real memory MCP server, then test it.

Which quest was your favorite, and what did the agent make?

Mission G — Design & Share (about 8 min)

Invent one more tool of your own, build it, then show the class your coolest creation.

My tool's name:

What does it do, and when should the agent use it (its docstring)?

Write the mission prompt you'll give the agent to build it:

Which creation are you sharing with the class, and why?

Wrap-Up — Reflect (about 4 min)

Was it cooler to build a TOOL or to give the agent MEMORY? Why?

If you could build your agent any tool in the world, what would it be?